

Homework — 2.3.1 Algorithms

Name: _____ Date: _____ Mark: _____ / 25

OCR H446 — set after the 2.3.1 lesson. Show traces/working. Code uses OCR Exam Reference Language.

Part A — This week's topic (~14 marks)

1. Complete the table with the **worst-case Big O time complexity** of each algorithm and a one-line **justification** for each. [2] (AO1/AO2)

Algorithm	Worst-case time complexity (Big O)	Justification
Linear search		
Binary search		
Bubble sort		
Merge sort		

2. Trace a **bubble sort** (one pass = compare each adjacent pair left-to-right, swapping if out of order) on the list below. Complete the table to show the list **after each complete pass** until it is sorted. [3] (AO2)

Start: [6, 2, 9, 4, 1]

After pass	Item 1	Item 2	Item 3	Item 4	Item 5
Pass 1					
Pass 2					
Pass 3					
Pass 4					

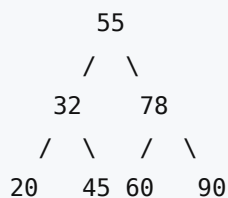
3. The function below is intended to perform a **binary search**, returning the index of `target` in the sorted 9-item array `numbers` (indices 0 to 8), or `-1` if it is not present. It contains **three** errors. Identify each error and write the corrected line. [3] (AO2/AO3)

```

function binarySearch(numbers, target)
  low = 0
  high = 8
  while low < high
    mid = (low + high) DIV 2
    if numbers[mid] == target then
      return mid
    elseif numbers[mid] < target then
      low = mid
    else
      high = mid
    endif
  endwhile
  return -1
endfunction

```

4. Consider the binary search tree below. Write the letter of the correct output order in the Match column for each traversal. One output order is not needed. [2] (AO2)



Traversal	Match
Depth-first (post-order)	
Breadth-first	
In-order (left subtree → node → right subtree)	

Letter	Output order
A	55, 32, 20, 45, 78, 60, 90
B	20, 45, 32, 60, 90, 78, 55
C	20, 32, 45, 55, 60, 78, 90
D	55, 32, 78, 20, 45, 60, 90

5. A **merge sort** is performed on the list [8, 3, 5, 2]. The five steps below are listed out of order. Number them 1–5. [2] (AO2)

Step	Order (1–5)
Merge [3, 8] and [2, 5] to give [2, 3, 5, 8]	
Split [8, 3, 5, 2] into [8, 3] and [5, 2]	
Merge [8] and [3] to give [3, 8]	
Split [8, 3] and [5, 2] into the single-item lists [8], [3], [5], [2]	
Merge [5] and [2] to give [2, 5]	

6. **Dijkstra's algorithm** is run on the weighted graph below, starting at node **A**, to find the shortest distance to every other node.

Edges (undirected, with weights): A–B = 2, A–C = 5, B–C = 1, B–D = 7, C–D = 3, C–E = 8, D–E = 2

Complete the table of **final shortest distances** from A. Show your working. **[2]** (AO2)

Node	Shortest distance from A	Via (previous node)
A	0	—
B		
C		
D		
E		

Part B — Retrieval: keep it fresh (~6 marks)

7. (2.2.2) A problem can only be solved in **exponential time or worse**. State the term used to describe such a problem, and give **one** example. **[2]**

8. (2.2.1) State **one** advantage and **one** disadvantage of implementing a repeated process using **recursion** rather than **iteration**. **[2]**

9. (2.1.5) State **one** benefit and **one** drawback of processing the parts of a problem **concurrently**. **[2]**

Part C — Exam-style question (~5 marks)

10. A logistics company needs to calculate the shortest delivery distance from a single depot to **every** delivery point on a road network where all distances are positive.

Recommend either **Dijkstra's algorithm** or the **A*** algorithm for this task, and **justify** your choice by comparing the two. **[5]** (AO3)
